

Hamiltonian Cycles and Paths

Przemysław Gordinowicz

Institute of Mathematics,
Łódź University of Technology



Graphs & Social Networks
Lecture 7



Outline

- 1 Hamiltonian Cycles and Paths
- 2 Necessary and Sufficient Conditions
- 3 Travelling Salesman and Chinese Postman
- 4 *NP*-completeness
- 5 Approximation algorithms for the travelling salesman problem

Outline

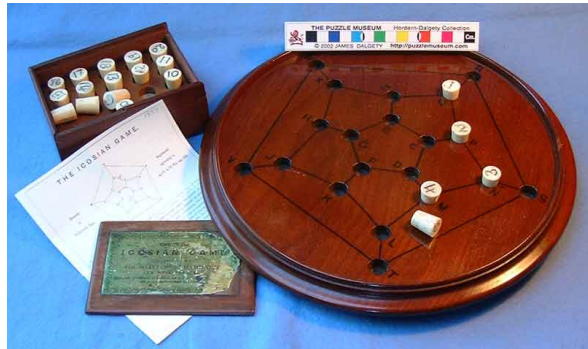


- 1 Hamiltonian Cycles and Paths
- 2 Necessary and Sufficient Conditions
- 3 Travelling Salesman and Chinese Postman
- 4 NP-completeness
- 5 Approximation algorithms for the travelling salesman problem

Hamilton's Puzzle



William R. Hamilton
(1805–1865)



„Icosian game” [source](#) [\[link\]](#)

The first player defines a 5-vertex path, the task of the second player is to extend it to a cycle traversing all the vertices.

Hamilton's Puzzle



William R. Hamilton
(1805–1865)



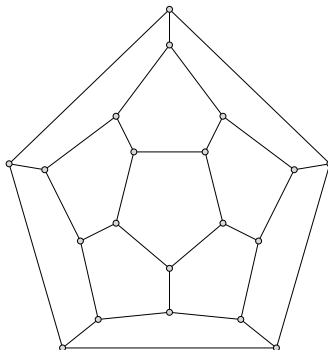
„Traveller's Dodecahedron" [source \[link\]](#)

The first player defines a 5-vertex path, the task of the second player is to extend it to a cycle traversing all the vertices.

Hamilton's Puzzle



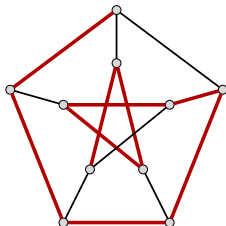
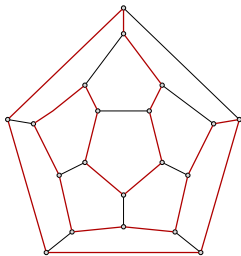
William R. Hamilton
(1805–1865)



„Traveller's Dodecahedron”

The first player defines a 5-vertex path, the task of the second player is to extend it to a cycle traversing all the vertices.

Definition



Definition

A **Hamiltonian cycle** in a graph is a spanning cycle, i.e. a cycle containing all the vertices of the graph. A graph with a Hamiltonian cycle is called a **Hamiltonian graph**.

A **Hamiltonian path** in a graph is a spanning path. A graph with a Hamiltonian path is called a **semi-Hamiltonian graph**.



No characterization

No characterization of Hamiltonian graphs has been found yet — there are only partial results.

Moreover, it is not known whether there is a polynomial time algorithm to determine whether a graph is Hamiltonian. Deciding whether a given graph G is Hamiltonian is one of the *NP*-complete problems.

Observation

The complete graph K_n is Hamiltonian for $n \geq 3$.
Adding edges to a Hamiltonian (or semi-Hamiltonian) graph gives a Hamiltonian (or semi-Hamiltonian) graph.



No characterization

No characterization of Hamiltonian graphs has been found yet — there are only partial results.

Moreover, it is not known whether there is a polynomial time algorithm to determine whether a graph is Hamiltonian. Deciding whether a given graph G is Hamiltonian is one of the *NP*-complete problems.

Observation

The complete graph K_n is Hamiltonian for $n \geq 3$.
Adding edges to a Hamiltonian (or semi-Hamiltonian) graph gives a Hamiltonian (or semi-Hamiltonian) graph.

Outline



- 1 Hamiltonian Cycles and Paths
- 2 Necessary and Sufficient Conditions**
- 3 Travelling Salesman and Chinese Postman
- 4 NP-completeness
- 5 Approximation algorithms for the travelling salesman problem



Observation

Let G be an undirected Hamiltonian graph. Then:

- ➊ G is connected (even 2-connected).
- ➋ For any $v \in V(G)$, $\deg(v) \geq 2$.
- ➌ All edges incident to vertices of degree 2 in G are part of a Hamiltonian cycle.



Observation

Let G be an undirected Hamiltonian graph. Then:

- ➊ G is connected (even 2-connected).
- ➋ For any $v \in V(G)$, $\deg(v) \geq 2$.
- ➌ All edges incident to vertices of degree 2 in G are part of a Hamiltonian cycle.

Remark

“Essential” edges.



Observation

Let G be an undirected Hamiltonian graph. Then:

- ➊ G is connected (even 2-connected).
- ➋ For any $v \in V(G)$, $\deg(v) \geq 2$.
- ➌ All edges incident to vertices of degree 2 in G are part of a Hamiltonian cycle.

Observation

Let G be an undirected Hamiltonian graph. Then for every nonempty set $S \subseteq V(G)$ the graph $G \setminus S$ has at most $|S|$ connected components.

Bipartite graphs

Theorem

Let G be a bipartite graph such that $V(G) = A \cup B$, $A \cap B = \emptyset$ and $E(G) \subseteq \{\{u, v\} : u \in A \wedge v \in B\}$. Then:

- 1 If G is semi-Hamiltonian then $||A| - |B|| \leq 1$.
- 2 If G is Hamiltonian then $|A| = |B|$.

Ore's Theorem

Lemma (Ore, 1960)

Let G be an undirected graph. If $u, v \in V(G)$ are distinct vertices such that $uv \notin E(G)$ and $\deg(u) + \deg(v) \geq |V(G)|$ then G is Hamiltonian if and only if $(V(G), E(G) \cup \{uv\})$ is Hamiltonian.

Theorem (Ore, 1960)

Let G be an undirected graph, $|V(G)| \geq 3$. If for any non-adjacent vertices $u, v \in V(G)$ we have

$$\deg(u) + \deg(v) \geq |V(G)|$$

then G is Hamiltonian.

Ore's Theorem



Lemma (Ore, 1960)

Let G be an undirected graph. If $u, v \in V(G)$ are distinct vertices such that $uv \notin E(G)$ and $\deg(u) + \deg(v) \geq |V(G)|$ then G is Hamiltonian if and only if $(V(G), E(G) \cup \{uv\})$ is Hamiltonian.

Theorem (Ore, 1960)

Let G be an undirected graph, $|V(G)| \geq 3$. If for any non-adjacent vertices $u, v \in V(G)$ we have

$$\deg(u) + \deg(v) \geq |V(G)|$$

then G is Hamiltonian.

Around Ore's theorem



Theorem (Dirac, 1952)

Let G be an undirected graph, $|V(G)| \geq 3$. If $\delta(G) \geq |V(G)|/2$ then G is Hamiltonian.

Definition

Hamiltonian closure of G is a graph on $V(G)$ obtained from G by adding edges according to the rule:

$$\deg(u) + \deg(v) \geq |V(G)| \rightarrow uv \in E, \text{ for } u, v \in V(G).$$



Around Ore's theorem

Theorem (Dirac, 1952)

Let G be an undirected graph, $|V(G)| \geq 3$. If $\delta(G) \geq |V(G)|/2$ then G is Hamiltonian.

Definition

Hamiltonian closure of G is a graph on $V(G)$ obtained from G by adding edges according to the rule:

$$\deg(u) + \deg(v) \geq |V(G)| \rightarrow uv \in E, \text{ for } u, v \in V(G).$$

Remark

The order in which the edges are added does not matter.

Around Ore's theorem



Theorem (Dirac, 1952)

Let G be an undirected graph, $|V(G)| \geq 3$. If $\delta(G) \geq |V(G)|/2$ then G is Hamiltonian.

Definition

Hamiltonian closure of G is a graph on $V(G)$ obtained from G by adding edges according to the rule:

$$\deg(u) + \deg(v) \geq |V(G)| \rightarrow uv \in E, \text{ for } u, v \in V(G).$$

Theorem (Bondy, Chvátal, 1976)

An undirected graph G is Hamiltonian if and only if its closure is Hamiltonian.

Outline

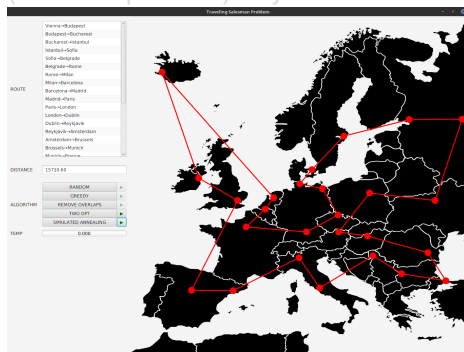


- 1 Hamiltonian Cycles and Paths
- 2 Necessary and Sufficient Conditions
- 3 Travelling Salesman and Chinese Postman
- 4 NP-completeness
- 5 Approximation algorithms for the travelling salesman problem

Travelling Salesman Problem (TSP)

A travelling salesman has to visit n cities and return back to his home/company. Knowing the distances between the cities, determine the shortest route for the travelling salesman.

In the language of graph theory, the problem is formulated in several (almost equivalent) ways.

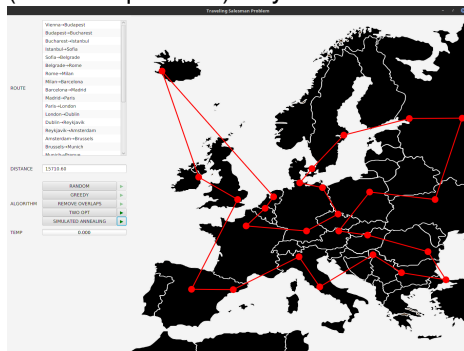


source towardsdatascience.com

Travelling Salesman Problem (TSP)

A travelling salesman has to visit n cities and return back to his home/company. Knowing the distances between the cities, determine the shortest route for the travelling salesman.

In the language of graph theory, the problem is formulated in several (almost equivalent) ways.



source towardsdatascience.com



Travelling Salesman Problem (TSP)

A travelling salesman has to visit n cities and return back to his home/company. Knowing the distances between the cities, determine the shortest route for the travelling salesman.

In the language of graph theory, the problem is formulated in several (almost equivalent) ways.

Problem

In a complete graph with weights, find a Hamiltonian cycle with the smallest weight.

This will be the basic formulation of the problem.

Since the graph is complete, only the *weight matrix* is important. The graph can be directed or undirected (the *asymmetric* or *symmetric* problem).



Travelling Salesman Problem (TSP)

A travelling salesman has to visit n cities and return back to his home/company. Knowing the distances between the cities, determine the shortest route for the travelling salesman.

In the language of graph theory, the problem is formulated in several (almost equivalent) ways.

Problem

In a weighted, connected graph, find a Hamiltonian cycle with minimum weight.

With this formulation of the problem, the existence of a feasible solution is not guaranteed.

It can be relatively easily reduced to the previous one.



Travelling Salesman Problem (TSP)

A travelling salesman has to visit n cities and return back to his home/company. Knowing the distances between the cities, determine the shortest route for the travelling salesman.

In the language of graph theory, the problem is formulated in several (almost equivalent) ways.

Problem

In a weighted, connected graph, find a closed walk with minimum weight passing through all vertices.

Sensible from the application perspective. It reduces very easily to the next formulation.

Travelling Salesman Problem (TSP)



A travelling salesman has to visit n cities and return back to his home/company. Knowing the distances between the cities, determine the shortest route for the travelling salesman.

In the language of graph theory, the problem is formulated in several (almost equivalent) ways.

Problem

In a complete graph with weights satisfying the triangle inequality, find the Hamiltonian cycle with the smallest weight.

Having the triangle inequality satisfied we talk about the metric variant of the problem (metric TSP) (weights generate a metric, provided they all are positive).



Travelling Salesman Problem (TSP)

A travelling salesman has to visit n cities and return back to his home/company. Knowing the distances between the cities, determine the shortest route for the travelling salesman.

In the language of graph theory, the problem is formulated in several (almost equivalent) ways.

Problem

In a complete graph with weights, find the Hamiltonian cycle with the smallest weight.

This problem, apart from its many practical applications (logistics, but also e.g. DNA sequencing, or electronic circuit design), has a very strong influence on the development of theoretical computer science — it is probably the most imaginative **NP-hard*** problem.



The Chinese Postman Problem

A postman has to walk all the streets in his district and return back to the post office. Knowing the lengths of the streets, determine the shortest route for the postman.

In the language of graph theory, the problem is formulated as follows.

Problem (Mei-Ku Kuan, 1962)

In a weighted, connected graph, find a closed walk of minimum weight that passes through all edges.

The problem is easy when the graph is Eulerian or semi-Eulerian (has an Eulerian path). It has a polynomial time algorithm, except when considering mixed graphs (some streets are one-way, some are not).



The Chinese Postman Problem

A postman has to walk all the streets in his district and return back to the post office. Knowing the lengths of the streets, determine the shortest route for the postman.

In the language of graph theory, the problem is formulated as follows.

Problem (Mei-Ku Kuan, 1962)

In a weighted, connected graph, find a closed walk of minimum weight that passes through all edges.

The problem is easy when the graph is Eulerian or semi-Eulerian (has an Eulerian path). It has a polynomial time algorithm, except when considering mixed graphs (some streets are one-way, some are not).



The Chinese Postman Problem

A postman has to walk all the streets in his district and return back to the post office. Knowing the lengths of the streets, determine the shortest route for the postman.

In the language of graph theory, the problem is formulated as follows.

Problem (Mei-Ku Kuan, 1962)

In a weighted, connected graph, find a closed walk of minimum weight that passes through all edges.

The problem is easy when the graph is Eulerian or semi-Eulerian (has an Eulerian path). It has a polynomial time algorithm, except when considering mixed graphs (some streets are one-way, some are not).



Outline

- 1 Hamiltonian Cycles and Paths
- 2 Necessary and Sufficient Conditions
- 3 Travelling Salesman and Chinese Postman
- 4 NP-completeness**
- 5 Approximation algorithms for the travelling salesman problem



Polynomial time — why

How to calculate computational complexity?

What do we actually count?

Most algorithms that we implement in practice have polynomial running time (i.e. computational complexity of the order of $O(n^k)$, where n is the *size* of the input data).

We will say that problems that have polynomial algorithms are *easy*.

Polynomial time — why



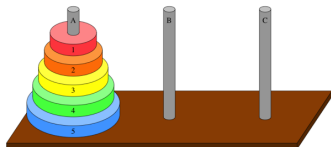
How to calculate computational complexity?

What do we actually count?

Most algorithms that we implement in practice have polynomial running time (i.e. computational complexity of the order of $O(n^k)$, where n is the *size* of the input data).

We will say that problems that have polynomial algorithms are *easy*.

Exponential time or worse



source en.khanacademy.org

The Towers of Hanoi problem with n disks requires $2^n - 1$ translations.

$$(n + 1)!$$

Writing out all the permutations of the n -element set will take $O((n + 1)!)$ time, or $\Theta((\frac{n}{e})^n)$ (Stirling's formula).



Decision problems

The previous examples required writing out a huge result. That's one reason we'll focus now exclusively on *decision problems* — giving answers TRUE or FALSE. Instead of, for example, determining the shortest path in a graph (optimization problem $\text{SHORTESTPATH}(G, w, s, t)$), we'll check whether there exists a path of length at most k (decision problem $k\text{PATH}(G, w, s, t, k)$).

Towers of Hanoi are no longer a problem!

Decision problems



The previous examples required writing out a huge result. That's one reason we'll focus now exclusively on *decision problems* — giving answers TRUE or FALSE. Instead of, for example, determining the shortest path in a graph (optimization problem $\text{SHORTESTPATH}(G, w, s, t)$), we'll check whether there exists a path of length at most k (decision problem $k\text{PATH}(G, w, s, t, k)$).

Towers of Hanoi are no longer a problem!

Decision problems

The previous examples required writing out a huge result. That's one reason we'll focus now exclusively on *decision problems* — giving answers TRUE or FALSE. Instead of, for example, determining the shortest path in a graph (optimization problem $\text{SHORTESTPATH}(G, w, s, t)$), we'll check whether there exists a path of length at most k (decision problem $k\text{PATH}(G, w, s, t, k)$).



The problem of existence of a winning strategy in checkers ($n \times n$) given the situation on the board will still require superexponential time — complexity $\Omega(n^n)$.

Coding, formal languages



In complexity theory, problems with properly **encoded** input data are considered — the input is given in the form of finite sequences of zeros and ones (or words over another, finite alphabet) — so-called concrete problems.

The input data for an algorithm that solves a decision problem is an encoded form of a problem instance. We say that the algorithm **solves** a concrete problem in time $O(T(n))$ if, given a problem instance (word) of length n , it can compute the solution in time $O(T(n))$.

We can therefore define *complexity class* P as the set of specific decision problems solvable in polynomial time (complexity $O(n^k)$ for some $k \in \mathbb{N}$). Note — different encodings.



Coding, formal languages

In complexity theory, problems with properly **encoded** input data are considered — the input is given in the form of finite sequences of zeros and ones (or words over another, finite alphabet) — so-called concrete problems.

The input data for an algorithm that solves a decision problem is an encoded form of a problem instance. We say that the algorithm **solves** a concrete problem in time $O(T(n))$ if, given a problem instance (word) of length n , it can compute the solution in time $O(T(n))$.

We can therefore define *complexity class* P as the set of specific decision problems solvable in polynomial time (complexity $O(n^k)$ for some $k \in \mathbb{N}$).
Note — different encodings.

Coding, formal languages



In complexity theory, problems with properly **encoded** input data are considered — the input is given in the form of finite sequences of zeros and ones (or words over another, finite alphabet) — so-called concrete problems.

The input data for an algorithm that solves a decision problem is an encoded form of a problem instance. We say that the algorithm **solves** a concrete problem in time $O(T(n))$ if, given a problem instance (word) of length n , it can compute the solution in time $O(T(n))$.

We can therefore define *complexity class* P as the set of specific decision problems solvable in polynomial time (complexity $O(n^k)$ for some $k \in \mathbb{N}$).

Note — different encodings.

Coding, formal languages



In complexity theory, problems with properly **encoded** input data are considered — the input is given in the form of finite sequences of zeros and ones (or words over another, finite alphabet) — so-called concrete problems.

The input data for an algorithm that solves a decision problem is an encoded form of a problem instance. We say that the algorithm **solves** a concrete problem in time $O(T(n))$ if, given a problem instance (word) of length n , it can compute the solution in time $O(T(n))$.

We can therefore define *complexity class* P as the set of specific decision problems solvable in polynomial time (complexity $O(n^k)$ for some $k \in \mathbb{N}$). Note — different encodings.



Coding, formal languages

In complexity theory, problems with properly **encoded** input data are considered — the input is given in the form of finite sequences of zeros and ones (or words over another, finite alphabet) — so-called concrete problems.

The input data for an algorithm that solves a decision problem is an encoded form of a problem instance. We say that the algorithm **solves** a concrete problem in time $O(T(n))$ if, given a problem instance (word) of length n , it can compute the solution in time $O(T(n))$.

We can therefore define *complexity class* P as the set of specific decision problems solvable in polynomial time (complexity $O(n^k)$ for some $k \in \mathbb{N}$).

Lemma

Let Q be an abstract decision problem, e_1 and e_2 — polynomially equivalent encodings on the inputs of problem Q . Then $e_1(Q) \in P$ if $e_2(Q) \in P$.



Coding, formal languages

In complexity theory, problems with properly **encoded** input data are considered — the input is given in the form of finite sequences of zeros and ones (or words over another, finite alphabet) — so-called concrete problems.

The input data for an algorithm that solves a decision problem is an encoded form of a problem instance. We say that the algorithm **solves** a concrete problem in time $O(T(n))$ if, given a problem instance (word) of length n , it can compute the solution in time $O(T(n))$.

We can therefore define *complexity class* P as the set of specific decision problems solvable in polynomial time (complexity $O(n^k)$ for some $k \in \mathbb{N}$).

Remark

For the sake of time, we will omit a proper description using *formal languages*, appealing rather to intuition.



Coding, formal languages

In complexity theory, problems with properly **encoded** input data are considered — the input is given in the form of finite sequences of zeros and ones (or words over another, finite alphabet) — so-called concrete problems.

The input data for an algorithm that solves a decision problem is an encoded form of a problem instance. We say that the algorithm **solves** a concrete problem in time $O(T(n))$ if, given a problem instance (word) of length n , it can compute the solution in time $O(T(n))$.

We can therefore define *complexity class* P as the set of specific decision problems solvable in polynomial time (complexity $O(n^k)$ for some $k \in \mathbb{N}$).

Remark

We already know that there are problems that are undecidable in polynomial time. But there are also algorithmic problems that are undecidable at all!



Verification in polynomial time

Let us now stop to **verify** the solution of the problem. For example, for the problem k PATH, if we were also given a path of length k it would be a bit easier (but still in P).

What about **Hamiltonian cycle**? For the decision problem $\text{HAMIL}(G)$, deciding whether G is Hamiltonian, no efficient algorithm is known, but the problem becomes easy if we also specify the cycle.

Formally, the *verification algorithm* A for problem Q is a two-parameter algorithm $A(x, y)$, where the sequence x is the input to problem Q , and the second sequence y — is the *certificate* that confirms $Q(x)$.

It would be desirable for algorithm A to run in polynomial time and for the certificate y to be polynomial in size with respect to the size of x .



Verification in polynomial time

Let us now stop to **verify** the solution of the problem. For example, for the problem k PATH, if we were also given a path of length k it would be a bit easier (but still in P).

What about **Hamiltonian cycle**? For the decision problem $\text{HAMIL}(G)$, deciding whether G is Hamiltonian, no efficient algorithm is known, but the problem becomes easy if we also specify the cycle.

Formally, the *verification algorithm* A for problem Q is a two-parameter algorithm $A(x, y)$, where the sequence x is the input to problem Q , and the second sequence y — is the *certificate* that confirms $Q(x)$.

It would be desirable for algorithm A to run in polynomial time and for the certificate y to be polynomial in size with respect to the size of x .



Verification in polynomial time

Let us now stop to **verify** the solution of the problem. For example, for the problem k PATH, if we were also given a path of length k it would be a bit easier (but still in P).

What about **Hamiltonian cycle**? For the decision problem $\text{HAMIL}(G)$, deciding whether G is Hamiltonian, no efficient algorithm is known, but the problem becomes easy if we also specify the cycle.

Formally, the *verification algorithm* A for problem Q is a two-parameter algorithm $A(x, y)$, where the sequence x is the input to problem Q , and the second sequence y — is the *certificate* that confirms $Q(x)$.

It would be desirable for algorithm A to run in polynomial time and for the certificate y to be polynomial in size with respect to the size of x .



Verification in polynomial time

Let us now stop to **verify** the solution of the problem. For example, for the problem k PATH, if we were also given a path of length k it would be a bit easier (but still in P).

What about **Hamiltonian cycle**? For the decision problem $\text{HAMIL}(G)$, deciding whether G is Hamiltonian, no efficient algorithm is known, but the problem becomes easy if we also specify the cycle.

Formally, the *verification algorithm* A for problem Q is a two-parameter algorithm $A(x, y)$, where the sequence x is the input to problem Q , and the second sequence y — is the *certificate* that confirms $Q(x)$.

It would be desirable for algorithm A to run in polynomial time and for the certificate y to be polynomial in size with respect to the size of x .

Complexity class NP — easily verifiable problems

Complexity class NP (nondeterministic polynomial time) is the set of algorithmic problems that can be verified by a polynomial algorithm. That is, the problem $Q \in NP$ if there exists a polynomial algorithm A and constant c such that for any input x :

$Q(x) \Leftrightarrow$ there is such a certificate y , $|y| \in O(|x|^c)$, that $A(x, y) = \text{TRUE}$.

Question

Is $P = NP$?

Fundamental problem in the theory of computer science.

Definition

$coNP = \{\bar{L} : L \in NP\}$.

It is known that $P \subseteq NP \cap coNP$.

Question

Does $coNP = NP$?

Complexity class NP — easily verifiable problems

Complexity class NP (nondeterministic polynomial time) is the set of algorithmic problems that can be verified by a polynomial algorithm. That is, the problem $Q \in NP$ if there exists a polynomial algorithm A and constant c such that for any input x :

$Q(x) \Leftrightarrow$ there is such a certificate y , $|y| \in O(|x|^c)$, that $A(x, y) = \text{TRUE}$.

Question

Is $P = NP$?

Fundamental problem in the theory of computer science.

Definition

$coNP = \{\bar{L} : L \in NP\}$.

It is known that $P \subseteq NP \cap coNP$.

Question

Does $coNP = NP$?



Complexity class NP — easily verifiable problems

Complexity class NP (nondeterministic polynomial time) is the set of algorithmic problems that can be verified by a polynomial algorithm. That is, the problem $Q \in NP$ if there exists a polynomial algorithm A and constant c such that for any input x :

$Q(x) \Leftrightarrow$ there is such a certificate y , $|y| \in O(|x|^c)$, that $A(x, y) = \text{TRUE}$.

Question

Is $P = NP$?

Fundamental problem in the theory of computer science.

Definition

$coNP = \{\bar{L} : L \in NP\}$.

It is known that $P \subseteq NP \cap coNP$.

Question

Does $coNP = NP$?

Reducibility of (of NP problems) in polynomial time

The most important reason supporting the $P \neq NP$ conjecture is the existence of a class of *NP-complete* problems (e.g., the HAMIL problem), that is, such problems in the class NP that finding a polynomial-time algorithm to solve any of them would prove that $P = NP$ (i.e., that every NP problem has a polynomial-time algorithm).

Intuitively, the Q problem can be *reduced* to Q' if any instance of the Q problem can be reformulated as the Q' problem, from whose solution the solution to the Q problem can be read. So Q is *no harder* than Q' . A simple example: area of a quadrangle $\rightarrow$ area of a triangle.

Problem Q_1 is **reducible in polynomial time** to problem Q_2 , $Q_1 \leq_P Q_2$, if there exists a polynomial-time (with respect to the size of the argument) computable function $f: \text{data}Q_1 \rightarrow \text{data}Q_2$ such that for every $x \in \text{data}Q_1$ there is

$$Q_1(x) \Leftrightarrow Q_2(f(x)).$$

The function f is called the **reduction function**, and the

Reducibility of (of NP problems) in polynomial time

The most important reason supporting the $P \neq NP$ conjecture is the existence of a class of *NP-complete* problems (e.g., the HAMIL problem), that is, such problems in the class NP that finding a polynomial-time algorithm to solve any of them would prove that $P = NP$ (i.e., that every NP problem has a polynomial-time algorithm).

Intuitively, the Q problem can be *reduced* to Q' if any instance of the Q problem can be reformulated as the Q' problem, from whose solution the solution to the Q problem can be read. So Q is *no harder* than Q' . A simple example: area of a quadrangle $\rightarrow$ area of a triangle.

Problem Q_1 is **reducible in polynomial time** to problem Q_2 , $Q_1 \leq_P Q_2$, if there exists a polynomial-time (with respect to the size of the argument) computable function $f: \text{data}Q_1 \rightarrow \text{data}Q_2$ such that for every $x \in \text{data}Q_1$ there is

$$Q_1(x) \Leftrightarrow Q_2(f(x)).$$

The function f is called the **reduction function**, and the

Reducibility of (of NP problems) in polynomial time

The most important reason supporting the $P \neq NP$ conjecture is the existence of a class of *NP-complete* problems (e.g., the HAMIL problem), that is, such problems in the class NP that finding a polynomial-time algorithm to solve any of them would prove that $P = NP$ (i.e., that every NP problem has a polynomial-time algorithm).

Intuitively, the Q problem can be *reduced* to Q' if any instance of the Q problem can be reformulated as the Q' problem, from whose solution the solution to the Q problem can be read. So Q is *no harder* than Q' . A simple example: area of a quadrangle $\rightarrow$ area of a triangle.

Problem Q_1 is **reducible in polynomial time** to problem Q_2 , $Q_1 \leq_P Q_2$, if there exists a polynomial-time (with respect to the size of the argument) computable function $f: \text{data}Q_1 \rightarrow \text{data}Q_2$ such that for every $x \in \text{data}Q_1$ there is

$$Q_1(x) \Leftrightarrow Q_2(f(x)).$$

The function f is called the **reduction function**, and the



Reducibility of (of NP problems) in polynomial time

Problem Q_1 is **reducible in polynomial time** to problem Q_2 , $Q_1 \leq_P Q_2$, if there exists a polynomial-time (with respect to the size of the argument) computable function $f: \text{data}Q_1 \rightarrow \text{data}Q_2$ such that for every $x \in \text{data}Q_1$ there is

$$Q_1(x) \Leftrightarrow Q_2(f(x)).$$

The function f is called the **reduction function**, and the polynomial algorithm that computes it is F — the **reduction algorithm**.

Essential: $|f(x)| \in O(|x|^c)$ for some $c \in \mathbb{N}^+$.

Lemma

If Q_1, Q_2 are algorithmic problems such that $Q_1 \leq_P Q_2$ then from the fact that $Q_2 \in P$, it follows that $Q_1 \in P$.



Reducibility of (of NP problems) in polynomial time

Problem Q_1 is **reducible in polynomial time** to problem Q_2 , $Q_1 \leq_P Q_2$, if there exists a polynomial-time (with respect to the size of the argument) computable function $f: \text{data}Q_1 \rightarrow \text{data}Q_2$ such that for every $x \in \text{data}Q_1$ there is

$$Q_1(x) \Leftrightarrow Q_2(f(x)).$$

The function f is called the **reduction function**, and the polynomial algorithm that computes it is F — the **reduction algorithm**.

Essential: $|f(x)| \in O(|x|^c)$ for some $c \in \mathbb{N}^+$.

Lemma

If Q_1, Q_2 are algorithmic problems such that $Q_1 \leq_P Q_2$ then from the fact that $Q_2 \in P$, it follows that $Q_1 \in P$.

NP-hardness and NP-completeness

Definition

We say that the problem Q is **NP-complete**, denoting $Q \in NPC$, if

- 1 $Q \in NP$
- 2 $Q' \leq_P Q$ for all $Q' \in NP$

If the problem Q satisfies (2) then we say that it is **NP-hard**.

Theorem

If any NP-complete (NP-hard) problem has a polynomial solution, then $P = NP$. If any NP-complete problem has no polynomial solution, then no NP-complete (NP-hard) problem has one.



NP-hardness and NP-completeness

Definition

We say that the problem Q is **NP-complete**, denoting $Q \in NPC$, if

- 1 $Q \in NP$
- 2 $Q' \leq_P Q$ for all $Q' \in NP$

If the problem Q satisfies (2) then we say that it is **NP-hard**.

(Picture)

Theorem

If any NP-complete (NP-hard) problem has a polynomial solution, then $P = NP$. If any NP-complete problem has no polynomial solution, then no NP-complete (NP-hard) problem has one.



NP-hardness and NP-completeness

Definition

We say that the problem Q is **NP-complete**, denoting $Q \in NPC$, if

- 1 $Q \in NP$
- 2 $Q' \leq_P Q$ for all $Q' \in NP$

If the problem Q satisfies (2) then we say that it is *NP-hard*.

Theorem

If any NP-complete (NP-hard) problem has a polynomial solution, then $P = NP$. If any NP-complete problem has no polynomial solution, then no NP-complete (NP-hard) problem has one.



Satisfiability of logical formulas — SAT problem

It is important that $NPC \neq \emptyset$.

Computer scientists find it convenient to start with the CIRCUIT_SAT problem — the satisfiability of logical circuits (a circuit of logical gates is a simple model of a computer).

However, we will move directly to the SAT problem, the satisfiability of logical formulas. An instance of the (abstract) SAT problem is the logical formula ϕ constructed from:

- 1 logical variables $x_1, x_2, \dots$
- 2 logical connectives $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$
- 3 brackets

The **evaluation** of the formula ϕ is a vector of variable values, the **satisfying evaluation** is such that the formula takes the value TRUE. A formula with such a valuation is called **satisfiable**.

e.g. $\phi_1 = (x_1 \Rightarrow x_2) \wedge \neg x_1 \wedge (x_2 \vee x_1)$ is
and $\phi_2 = (x_1 \wedge x_2) \wedge (x_1 \Rightarrow \neg x_2)$ is not satisfiable.



Satisfiability of logical formulas — SAT problem

It is important that $NPC \neq \emptyset$.

Computer scientists find it convenient to start with the **CIRCUIT_SAT** problem — the satisfiability of logical circuits (a circuit of logical gates is a simple model of a computer).

However, we will move directly to the SAT problem, the satisfiability of logical formulas. An instance of the (abstract) SAT problem is the logical formula ϕ constructed from:

- 1 logical variables $x_1, x_2, \dots$
- 2 logical connectives $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$
- 3 brackets

The **evaluation** of the formula ϕ is a vector of variable values, the **satisfying evaluation** is such that the formula takes the value TRUE. A formula with such a valuation is called **satisfiable**.

e.g. $\phi_1 = (x_1 \Rightarrow x_2) \wedge \neg x_1 \wedge (x_2 \vee x_1)$ is
and $\phi_2 = (x_1 \wedge x_2) \wedge (x_1 \Rightarrow \neg x_2)$ is not satisfiable.



Satisfiability of logical formulas — SAT problem

It is important that $NPC \neq \emptyset$.

Computer scientists find it convenient to start with the **CIRCUIT_SAT** problem — the satisfiability of logical circuits (a circuit of logical gates is a simple model of a computer).

However, we will move directly to the **SAT** problem, the satisfiability of logical formulas. An instance of the (abstract) SAT problem is the logical formula ϕ constructed from:

- 1 logical variables $x_1, x_2, \dots$
- 2 logical connectives $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$
- 3 brackets

The **evaluation** of the formula ϕ is a vector of variable values, the **satisfying evaluation** is such that the formula takes the value TRUE. A formula with such a valuation is called **satisfiable**.

e.g. $\phi_1 = (x_1 \Rightarrow x_2) \wedge \neg x_1 \wedge (x_2 \vee x_1)$ is
and $\phi_2 = (x_1 \wedge x_2) \wedge (x_1 \Rightarrow \neg x_2)$ is not satisfiable.



Satisfiability of logical formulas — SAT problem

It is important that $NPC \neq \emptyset$.

Computer scientists find it convenient to start with the CIRCUI_T_SAT problem — the satisfiability of logical circuits (a circuit of logical gates is a simple model of a computer).

However, we will move directly to the SAT problem, the satisfiability of logical formulas. An instance of the (abstract) SAT problem is the logical formula ϕ constructed from:

- 1 logical variables $x_1, x_2, \dots$
- 2 logical connectives $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$
- 3 brackets

The evaluation of the formula ϕ is a vector of variable values, the **satisfying evaluation** is such that the formula takes the value TRUE. A formula with such a valuation is called **satisfiable**.

e.g. $\phi_1 = (x_1 \Rightarrow x_2) \wedge \neg x_1 \wedge (x_2 \vee x_1)$ is
and $\phi_2 = (x_1 \wedge x_2) \wedge (x_1 \Rightarrow \neg x_2)$ is not satisfiable.



Satisfiability of logical formulas — SAT problem

It is important that $NPC \neq \emptyset$.

Computer scientists find it convenient to start with the **CIRCUIT_SAT** problem — the satisfiability of logical circuits (a circuit of logical gates is a simple model of a computer).

However, we will move directly to the **SAT** problem, the satisfiability of logical formulas. An instance of the (abstract) SAT problem is the logical formula ϕ constructed from:

- 1 logical variables $x_1, x_2, \dots$
- 2 logical connectives $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$
- 3 brackets

The evaluation of the formula ϕ is a vector of variable values, the **satisfying evaluation** is such that the formula takes the value TRUE. A formula with such a valuation is called **satisfiable**.

e.g. $\phi_1 = (x_1 \Rightarrow x_2) \wedge \neg x_1 \wedge (x_2 \vee x_1)$ is

and $\phi_2 = (x_1 \wedge x_2) \wedge (x_1 \Rightarrow \neg x_2)$ is not satisfiable.

SAT



Observation

$SAT \in NP$.

Obvious.

By the way: $TAUTOLOGY \in coNP$.

Theorem (Cook (1971), Levin (1973))

$SAT \in NPC$.

One has to show $Q \leq_P SAT$ for any $Q \in NP$.

It is enough to meticulously check the state of computer memory for the polynomial algorithm A verifying the problem Q .

SAT



Observation

$SAT \in NP$.

Obvious.

Theorem (Cook (1971), Levin (1973))

$SAT \in NPC$.

One has to show $Q \leq_P SAT$ for any $Q \in NP$.

It is enough to meticulously check the state of computer memory for the polynomial algorithm A verifying the problem Q .

SAT



Observation

$SAT \in NP$.

Obvious.

Theorem (Cook (1971), Levin (1973))

$SAT \in NPC$.

One has to show $Q \leq_P SAT$ for any $Q \in NP$.

It is enough to meticulously check the state of computer memory for the polynomial algorithm A verifying the problem Q .



SAT and 3SAT, or simplifying the language

A **literal** in a logical formula is an occurrence of a variable or its negation. We will say that a logical formula is in **conjunctive normal form (CNF)** if it is written as a conjunction of clauses (being disjunctions of one or more literals).

A formula is in **3-conjunctive normal form (3CNF)** if each clause contains exactly 3 different literals.

e.g. $(x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_2 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee x_3 \vee \neg x_4)$.

$3SAT = \{\langle \phi \rangle : \phi \text{ is a satisfiable logical formula of the form 3CNF}\}.$

Theorem (Karp (21 NP-complete problems, 1972))

$3SAT \in NPC.$

Proof. It suffices to show that $SAT \leq_P 3SAT$.



SAT and 3SAT, or simplifying the language

A **literal** in a logical formula is an occurrence of a variable or its negation. We will say that a logical formula is in **conjunctive normal form (CNF)** if it is written as a conjunction of clauses (being disjunctions of one or more literals).

A formula is in **3-conjunctive normal form (3CNF)** if each clause contains exactly 3 different literals.

e.g. $(x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_2 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee x_3 \vee \neg x_4)$.

$3SAT = \{\langle \phi \rangle : \phi \text{ is a satisfiable logical formula of the form 3CNF}\}$.

Theorem (Karp (21 NP-complete problems, 1972))

$3SAT \in NPC$.

Proof. It suffices to show that $SAT \leq_P 3SAT$.



SAT and 3SAT, or simplifying the language

A **literal** in a logical formula is an occurrence of a variable or its negation. We will say that a logical formula is in **conjunctive normal form (CNF)** if it is written as a conjunction of clauses (being disjunctions of one or more literals).

A formula is in **3-conjunctive normal form (3CNF)** if each clause contains exactly 3 different literals.

e.g. $(x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_2 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee x_3 \vee \neg x_4)$.

$3SAT = \{\langle \phi \rangle : \phi \text{ is a satisfiable logical formula of the form 3CNF}\}$.

Theorem (Karp (21 NP-complete problems, 1972))

$3SAT \in NPC$.

Proof. It suffices to show that $SAT \leq_P 3SAT$.

NP-completeness of the Hamiltonian cycle problem



Theorem (Karp (1972))

$HAMIL \in NPC$.

Proof. $3SAT \leq_P HAMIL$.

NP-completeness of the Hamiltonian cycle problem

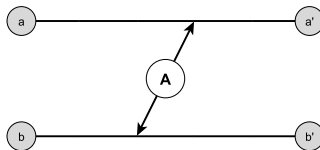
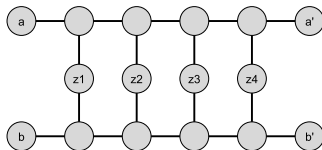


Theorem (Karp (1972))

$HAMIL \in NPC$.

Proof. $3SAT \leq_P HAMIL$.

Gadgets.



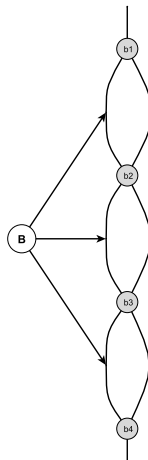
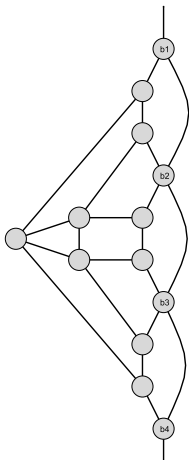
NP-completeness of the Hamiltonian cycle problem



Theorem (Karp (1972))

$HAMIL \in NPC$.

Proof. $3SAT \leq_P HAMIL$.



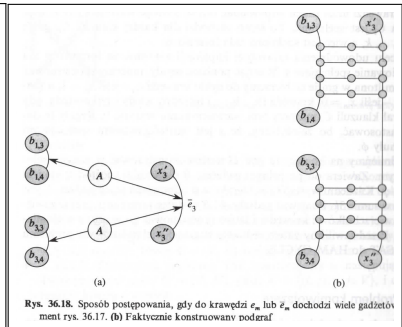
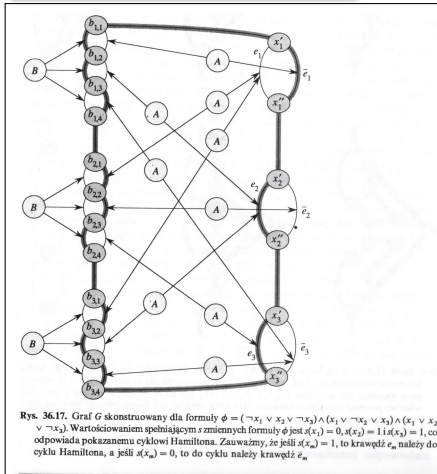
NP-completeness of the Hamiltonian cycle problem



Theorem (Karp (1972))

$HAMIL \in NPC$.

Proof. $3SAT \leq_P HAMIL$.





NP-hardness of the travelling salesman problem

Theorem

$kTSP \in NPC$.

Proof. $TSP \in NP$

$HAMIL \leq_P kTSP$.

We consider the decision problem of the existence of a salesman route with total weight no greater than fixed $k \in \mathbb{R}$.

Let us consider the problem $HAMIL(G)$ on an arbitrary graph $G = (V, E)$. We construct a complete graph $G' = (V, E')$ with a cost function w given by

$$w(u, v) = \begin{cases} 0, & \text{gdy } uv \in E \\ 1, & \text{gdy } uv \notin E \end{cases}$$

Of course, G is Hamiltonian if G' has a travelling salesman cycle of weight 0.



NP-hardness of the travelling salesman problem

Theorem

$kTSP \in NPC$.

Proof. $TSP \in NP$

$HAMIL \leq_P kTSP$.

Let us consider the problem $HAMIL(G)$ on an arbitrary graph $G = (V, E)$. We construct a complete graph $G' = (V, E')$ with a cost function w given by

$$w(u, v) = \begin{cases} 0, & \text{gdy } uv \in E \\ 1, & \text{gdy } uv \notin E \end{cases}$$

Of course, G is Hamiltonian if G' has a travelling salesman cycle of weight 0.



NP-hardness of the travelling salesman problem

Theorem

$kTSP \in NPC$.

Proof. $TSP \in NP$

$HAMIL \leq_P kTSP$.

Let us consider the problem $HAMIL(G)$ on an arbitrary graph $G = (V, E)$. We construct a complete graph $G' = (V, E')$ with a cost function w given by

$$w(u, v) = \begin{cases} 0, & \text{gdy } uv \in E \\ 1, & \text{gdy } uv \notin E \end{cases}$$

Of course, G is Hamiltonian if G' has a travelling salesman cycle of weight 0.



NP-hardness of the travelling salesman problem

Theorem

$kTSP \in NPC$.

Proof. $TSP \in NP$
 $HAMIL \leq_P kTSP$.

Hence, TSP (optimization problem) is NP-hard.

Let us consider the problem HAMIL(G) on an arbitrary graph $G = (V, E)$. We construct a complete graph $G' = (V, E')$ with a cost function w given by

$$w(u, v) = \begin{cases} 0, & \text{gdy } uv \in E \\ 1, & \text{gdy } uv \notin E \end{cases}$$

Of course, G is Hamiltonian if G' has a travelling salesman cycle of weight 0.

Outline



- 1 Hamiltonian Cycles and Paths
- 2 Necessary and Sufficient Conditions
- 3 Travelling Salesman and Chinese Postman
- 4 NP-completeness
- 5 Approximation algorithms for the travelling salesman problem



How to handle hard problems?

For hard optimization problems, the following are considered:

- exact algorithms
- approximation algorithms

We say that an approximation algorithm has **relative constraint** $\alpha(n)$ if, for any input data of size n , the cost C of the solution given by the algorithm is estimated to within a factor of $\alpha(n)$ by the optimal cost C^*

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \alpha(n).$$

This makes sense when the partial costs are positive.



How to handle hard problems?

For hard optimization problems, the following are considered:

- exact algorithms
- approximation algorithms

We say that an approximation algorithm has **relative constraint** $\alpha(n)$ if, for any input data of size n , the cost C of the solution given by the algorithm is estimated to within a factor of $\alpha(n)$ by the optimal cost C^*

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \alpha(n).$$

This makes sense when the partial costs are positive.



Christofides' algorithm (1) for TSP with triangle inequality

ApproxMetricTSP1(G, w)

- ① build a minimum spanning tree T on (G, w)
- ② choose an arbitrary $r \in V(G)$ as the root of T
- ③ let L be a preorder list of vertices of the tree (T, r)
- ④ return a Hamiltonian cycle visiting vertices in order L

Theorem

The above algorithm is an approximation algorithm with relative constraint 2 for the travelling salesman problem with triangle inequality.



Christofides–Serdyukov's algorithm for metricTSP

ApproxMetricTSP2(G, w)

- ➊ build a minimum spanning tree T on (G, w)
- ➋ for a subgraph G induced by odd-degree vertices in T find the cheapest (with respect to w) perfect matching M
- ➌ compute an Eulerian walk in the multigraph H resulting from adding edges M to the tree T (this is the solution of Chinese Postman Problem on T !)
- ➍ return a Hamiltonian cycle resulting from removing repeated vertices in the Eulerian walk

Theorem

The above algorithm is an approximation algorithm with a relative constraint $\frac{3}{2}$ for metricTSP.



It should not work well without triangle inequality!

Theorem (Sahni, Gonzales [1976])

If $P \neq NP$ then for any constant $\alpha \geq 1$ there does not exist a polynomial approximation algorithm with relative bound α for the general travelling salesman problem (TSP).

Proof. Let $\alpha \geq 1$. Suppose that there is a α -approximation algorithm A . Consider the problem $\text{HAMIL}(G)$ on any graph $G = (V, E)$. We construct a complete graph $G' = (V, E')$ with a cost function w given by

$$w(u, v) = \begin{cases} 1, & \text{gdy } uv \in E \\ \alpha|V| + 1, & \text{gdy } uv \notin E \end{cases}$$

Of course, G is Hamiltonian if G' has a travelling salesman cycle of weight $|V|$. Algorithm $A(G')$ can then give a cycle of weight at most $\alpha|V|$. Hence, it cannot use edges outside E . But it means that $A(G')$ finds a Hamiltonian cycle in G . If A is a polynomial-time algorithm, then $P = NP$.



It should not work well without triangle inequality!

Theorem (Sahni, Gonzales [1976])

If $P \neq NP$ then for any constant $\alpha \geq 1$ there does not exist a polynomial approximation algorithm with relative bound α for the general travelling salesman problem (TSP).

Proof. Let $\alpha \geq 1$. Suppose that there is a α -approximation algorithm A . Consider the problem $\text{HAMIL}(G)$ on any graph $G = (V, E)$. We construct a complete graph $G' = (V, E')$ with a cost function w given by

$$w(u, v) = \begin{cases} 1, & \text{gdy } uv \in E \\ \alpha|V| + 1, & \text{gdy } uv \notin E \end{cases}$$

Of course, G is Hamiltonian if G' has a travelling salesman cycle of weight $|V|$. Algorithm $A(G')$ can then give a cycle of weight at most $\alpha|V|$. Hence, it cannot use edges outside E . But it means that $A(G')$ finds a Hamiltonian cycle in G . If A is a polynomial-time algorithm, then $P = NP$.



It should not work well without triangle inequality!

Theorem (Sahni, Gonzales [1976])

If $P \neq NP$ then for any constant $\alpha \geq 1$ there does not exist a polynomial approximation algorithm with relative bound α for the general travelling salesman problem (TSP).

Proof. Let $\alpha \geq 1$. Suppose that there is a α -approximation algorithm A . Consider the problem $\text{HAMIL}(G)$ on any graph $G = (V, E)$. We construct a complete graph $G' = (V, E')$ with a cost function w given by

$$w(u, v) = \begin{cases} 1, & \text{gdy } uv \in E \\ \alpha|V| + 1, & \text{gdy } uv \notin E \end{cases}$$

Of course, G is Hamiltonian if G' has a travelling salesman cycle of weight $|V|$. Algorithm $A(G')$ can then give a cycle of weight at most $\alpha|V|$. Hence, it cannot use edges outside E . But it means that $A(G')$ finds a Hamiltonian cycle in G . If A is a polynomial-time algorithm, then $P = NP$.